RPC
2. RPC
什么是rcp
RPC是远程过程调用（Remote Procedure Call）的缩写。它是一种计算机通信协议，使得程序可以请求另一个进程或
者计算机上的服务，就像调用本地的函数一样，从而实现分布式系统之间的交互和通讯
RPC 主要是基于TCP/IP 协议的
RPC 是一种流行的进程间通信技术，用于构建客户端 - 服务器应用程序。
1984 年，Birrell 和 Nelson 在 ACM Transactions
on Computer Systems 期刊上发表了名为
“Implementing remote procedure calls”的论文，
该文对 RPC 的机制做了经典的诠释。RPC的概念和
实践比HTTP早
RPC 远程过程调用是指计算机 A 上的进程，调用
另外一台计算机 B 上的进程的方法。其中A 上面
的调用进程被挂起，而 B 上面的被调用进程开始
执行对应方法，并将结果返回给 A，计算机 A 接
收到返回值后，调用进程继续执行。
发起 RPC 的进程通过参数等方式将信息传送给
被调用方，然后被调用方处理结束后，再通过返
回值将信息传递给调用方。这一过程对于开发人
员来说是透明的，开发人员一般也无须知道双方
底层是如何进行消息通信和信息传递的，这样可
以让业务开发人员更专注于业务开发，而非底层
细节。
RPC 让程序之间的远程过程调用具有与本地调用类似的形式。
比如说某个程序需要读取某个文件的数据，开发人员会在代码中
执行 read 系统调用来获取数据。
当 read 实际是本地调用时，read 函数由链接器从依赖库中提取
出来，接着链接器会将它链接到该程序中。虽然 read 中执行了
特殊的系统调用，但它本身依然是通过将参数压入堆栈的常规方
式调用的，调用方并不知道 read 函数的具体实现和行为。
当 read 实际是一个远程过程时（比如调用远程文件服务器提供
的方法），调用方程序中需要引入 read 的接口定义，称为客户
端存根（client-stub）。远程过程 read 的客户端存根与本地
方法的 read 函数类似，都执行了本地函数调用。不同的是它底
层实现上不是进行操作系统调用读取本地文件来提供数据，而是
将参数打包成网络消息，并将此网络消息发送到远程服务器，交
由远程服务执行对应的方法，在发送完调用请求后，客户端存根
随即阻塞，直到收到服务器发回的响应消息为止。
为什么要用 RPC 就是无法在一个进程内, 甚至一个计算机内通过本地调用的方式完成的需求, 比如不同的系统间的通讯, 甚至不
同的组织间的通讯, 由于计算能力需要横向扩展, 需要在多台机器组成的集群上部署应用。
RPC框架的组成
一个完整的 RPC 框架包含了服务注册发现、负
载、容错、序列化、协议编码和网络传输等组
件。
不同的 RPC 框架包含的组件可能会有所不同，
但是一定都包含 RPC 协议相关的组件，RPC 协
议包括序列化、协议编解码器和网络传输栈
RPC 协议
分类
公有协议 HTTP、SMPP、WebService 等都是公有协议。
私有协议
如果是某个公司或者组织内部自定义、自己使用
的，没有被国际标准化组织接纳和认可的协议，
往往划为私有协议，例如 Thrift 协议和蚂蚁金服
的 Bolt 协议。
弊端，比如在通用性上、公网传输的能力上
高度定制化的私有协议可以最大限度地降低成
本，提升性能，提高灵活性与效率
协议设计上，需考虑三个关键问题
1. 协议包括的必要字段与主要业务负载字段。协议里设计的每
个字段都应该被使用到，避免无效字段
2. 通信功能特性的支持。比如，CRC 校验、安全校验、数据压
缩机制等
3. 协议的升级机制。毕竟是私有协议，没有长期的验证，字段
新增或者修改，是有可能发生的，因此升级机制是必须考虑的
一个RPC框架大致需要动态代理、序列化、网络请求、网络
请求接受(netty实现)、动态加载、反射这些知识点。
JDK 动态代理
Proxy 类
Proxy 类可以理解为动态创建代理类的工厂类，
它提供了一组静态方法和接口用于动态生成对象
和代理类。通常我们只需要使用
newProxyInstance() 方法
InvocationHandler 接口
JDK 动态代理所代理的对象必须实现一个或者多
个接口，生成的代理类也是接口的实现类，然后
通过 JDK 动态代理是通过反射调用的方式代理类
中的方法，不能代理接口中不存在的方法。
动态代理在 rpc中的应用
RPC 会自动给接口生成一个代理类，当我们在项
目中注入接口的时候，运行过程中实际绑定的是
这个接口生成的代理类。这样在接口方法被调用
的时候，它实际上是被生成代理类拦截到了，这
样我们就可以在生成的代理类里面，加入远程调
用逻辑。
RpcInvokerProxy 处理器必须要实现
InvocationHandler 接口的 invoke() 方法，被代
理的 RPC 接口在执行方法调用时，都会转发到
invoke() 方法上。invoke() 方法的核心流程主要
分为三步：构造 RPC 协议对象、发起 RPC 远程
调用、等待 RPC 调用执行结果。
核心原理如下
整个 RPC的核心功能
1. 客户端（服务消费端 Client）：调用远程方法的一端。
2. 服务端（服务提供端 Server）：提供远程方法的一端。
3.
客户端 Stub（桩）：这其实就是一代理类。代理类主
要做的事情很简单，就是把你调用方法、类、方法参数
等信息传递到服务端。
4.
服务端 Stub（桩）：这个桩就不是代理类了。这里的服
务端 Stub 实际指的就是接收到客户端执行方法的请求
后，去指定对应的方法然后返回结果给客户端的类。
5.
网络传输：网络传输就是把你调用的方法的信息比如说
参数这些东西传输到服务端，然后服务端执行完之后再
把返回结果通过网络传输给你传输回来。网络传输的实
现方式有很多种比如最基本的 Socket 或者性能以及封装
更加优秀的 Netty（推荐）。
调用过程时序图
1. 服务消费方（Client）以本地调用方式调用服务；
2.
Client Stub 接收到调用后，负责将方法、参数等组装成能
够进行网络传输的消息体 RpcRequest（需要进行序列
化）；
3. Client Stub 找到服务地址，并将消息发送到服务端；
4. Server Stub 收到消息体后，反序列化为 Java 对象
RpcRequest；
5. Server Stub 根据 RpcRequest 中的类、方法、方法参数等
信息调用本地的服务/方法；
6. 本地服务执行并将结果返回给 Server Stub；
7.
Server Stub 得到方法执行结果并将组装成能够进行网络传
输的消息体 RpcResponse（需要进行序列化）发送至消费
方；
8. Client Stub 接收到消息体，并反序列化为 Java 对象
RpcResponse 。这样，服务消费方得到了最终结果。
工作在哪层应用层协议
优缺点
RPC如何实现进程间通信
3. GPRC
历史
谷歌一直在使用一个名为 Stubby 的通用 RPC 框架来连接数千
个在多个数据中心运行并使用不同技术构建的微服务。
但是它与 Google 的内部基础架构耦合得太紧密
了，没有标准化，不能用作通用框架。
2015 年，Google 发布了开源 RPC 框架 gRPC，它是一个标准化的、通用的、跨平台的 RPC 基础设施。
什么是grpc gRPC是Google开源的一种高性能、通用的远程过程调用（RPC）框架，基于Protocol Buffers序列化协议进行数据传输。
设计目标在任何环境下运行，支持可插拔的负载均衡，跟踪，运行状况检查和身份验证。它不仅支持数据中心内部和跨数据中心的服务
调用，它也适用于分布式计算的最后一公里，将设备，移动应用程序和浏览器连接到后端服务。
GRPC设计的动机和原则 32
gRPC 特点
1. 语言中立，支持多种语言；
2. 基于 IDL (接口定义语言（Interface Define Language）)接口定义服务，通过 proto3
工具生成指定语言的数据结构、服务端接口以及客户端 Stub
3. 通信协议基于标准的 HTTP/2 设计，支持·双向流、消息头压缩、单 TCP 的多路复用、服
务端推送等特性，这些特性使得 gRPC 在移动端设备上更加省电和节省网络流量
4. 序列化支持 PB（Protocol Buffer）和 JSON，PB 是一种语言无关的高性能序列化框架，
基于 HTTP/2 + PB, 保障了 RPC 调用的高性能
术语 Protocol Buffers
什么是Protocol Buffers Protocol Buffers，是Google公司开发的一种数据描述语言，简称protobuf
作用 gRPC 使用protoc特殊的 gRPC 插件从 proto 文件生成代码：将获得生成的 gRPC 客户端和
服务器代码，以及用于填充、序列化和检索消息类型的常规协议缓冲区代码。
特点
1. 支持多种编程语言
2. 序列化数据体积小
3. 反序列化速度快
4. 序列化和反序列化代码自动生成
Protocol Buffers和gRPC什么关系
gRPC是RPC协议是一种实现，是一个框架；Protocol Buffers，是
Google公司开发的一种数据描述语言。
gRPC和Protocol Buffers的关系就好比浏览器和HTML的关系，不相互
依赖，但是需要相互配合使用，才能达到最好的效果。
Protocol Buffers基本语法
Protocol Buffers是一个带有.proto扩展名的普通文本文件。
协议缓冲区数据被构造为消息，其中每条消息都是一个小的信息
逻辑记录，包含一系列称为字段的键值对。
插件 protoc编译器通过插件机制实现对不同语言的支持
调用模型
五部分
Service：提供的服务
Client：gRPC客户端
gRPC Server：gRPC服务端接口
gRPC Stub：gRPC客户端接口
Proto Request/Proto Response(s) ：中间文件（代码/协议）
调用过程
1、客户端（gRPC Stub）调用 A 方法，发起 RPC 调用。
2、对请求信息使用 Protobuf 进行对象序列化压缩（IDL）。接口定义语言（Interface Define Language）
3、服务端（gRPC Server）接收到请求后，解码请求体，进行
业务逻辑处理并返回。
4、对响应结果使用 Protobuf 进行对象序列化压缩（IDL）。
5、客户端接受到服务端响应，解码请求体。回调被调用的 A 方
法，唤醒正在等待响应（阻塞）的客户端调用并返回响应结果
grpc 原理
框架架构 gRPC 采用分层架构实现
应用程序代码位于语言绑定之上。该应用层处理应用逻辑和数据编码逻辑。通常，开发
人员使用不同语言提供的编译器为数据编码逻辑生成源代码
最基础的一层是 gRPC Core 层，它从上层抽象了所有网络操作，以便应用程序开发人
员可以轻松地进行 RPC 开发。Core 层还提供核心功能的扩展，包括安全认证等
通信流程客户端与服务端是如何交互的客户端与服务端交互图
1. 在建立连接之前，客户端/服务端都会发送连接前言
（Magic+SETTINGS），确立协议和配置项。
2. 在传输数据时，是会涉及滑动窗口（WINDOW_UPDATE）
等流控策略的。
3. 传播 gRPC 附加信息时，是基于 HEADERS 帧进行传播和设
置；而具体的请求/响应数据是存储的 DATA 帧中的。
4. 请求/响应结果会分为 HTTP 和 gRPC 状态响应两种类型。
5. 客户端发起 PING，服务端就会回应 PONG，反之亦可。
编解码 Protocol buffer 使用不同的编码技术对不同类
型的数据进行编码
字符串值，则 Protocol buffer 使用 UTF-8 对该值进行编码
int32 字段类型的整数值，它使用一种称为 varints 的编码技术
四种通信方式
grpc提供了以下4种RPC类型
1. Unary：客户端向服务器发送单个请求并获
得单个响应，就像正常的函数调用一样。
消息结尾处设置流结束（EOS）标志
一旦 server 端发送带有状态详细信息的
Trailers 标头，通信就会结束。
2. 1
Server streaming：客户端发送请求后，服
务端可以返回多个响应
3. 1
Client streaming：客户端在一次交互中可
以发送多个请求，服务端处理完成后返回单
个响应
4. 1
Bidirectional Streaming：客户端可以发送
多个请求，服务端也可以返回多个响应
通信方式是如何实现
使用了哪些编码技术
使用了哪些底层的网络通信技术
操作
RPC超时
RPC 终止客户端和服务器都对调用的成功做出独立和本地的
判断，它们的结论可能不一致。
RPC取消客户端或服务器可以随时取消 RPC。RPC会立即响应
取消操作而终止，以便不再进行进一步的工作。
gRPC 的优势
1. 实现的进程间通信方式高效 gRPC 不使用 JSON 或 XML 等文本格式，而是使用基于二进制协议的 protocol buffer 与 gRPC 服务、客户
端进行通信。此外，gRPC 是在 HTTP/2 之上实现的 protocol buffer，这使得进程间通信更快。
2. 具有简单、定义良好的服务接口和协议你首先定义服务接口，然后处理实现细节。因此，与用于 RESTful 服务定义的 OpenAPI/Swagger 和用于
SOAP Web 服务的 WSDL 不同，gRPC 提供了简单但一致、可靠且可扩展的应用程序开发体验。
3. 强类型 protocol buffer 清楚地定义了应用程序之间通信的数据类型，这使得分布式应用程序开发更加稳定。因为静
态类型有助于减少你在构建跨多个团队和技术的云原生应用程序时遇到的大多数运行时和交互错误。
4. 支持多语言 gRPC 被设计成支持多种编程语言。使用 protocol buffer 的服务定义与语言无关。因此，你可以选择 grpc
支持的任意语言，并与任何现有的 gRPC 服务或客户端进行通信。
5. 支持双向流式传输 gRPC 对客户端或服务器端流式传输具有原生支持，这使得开发流媒体服务或流媒体客户端变得更加容易。
6. 内置多种高级特性 gRPC 提供对高级特性的内置支持，例如身份验证、加密、元数据交换、压缩、负载平衡、服务发现等。
7. 与云原生生态系统高度集成 gRPC 是 CNCF 的一部分，大多数现代框架和技术都为 gRPC 提供了开箱即用的原生支持。例如，Envoy 等
CNCF 下的许多项目都支持使用 gRPC 作为通信协议
gRPC 应用的一个示例
1. 首先，你需要定义一个服务接口。服务接口定义包含有关客户端如何使用你的服务、允许客户端远程调用哪些方
法、调用这些方法时需要传递哪些参数、返回格式是怎么样的等信息。一般通过 protocol buffer 来定义服务接口
2.
使用 protocol buffer 定义好服务接口之后，你可以使用它提供的 protoc 工具生成被称为服务器骨架 (Server
Skeleton) 的服务端代码，它通过提供低级通信抽象来简化服务器端逻辑。此外，你还可以生成客户端代码，称为
客户端存根 (client stub)，它通过抽象来简化客户端通信，以隐藏不同编程语言的低级通信
3. 客户端使用你在服务接口定义中指定的方法来进行远程调用，就像进行本地函数调用一样简单。底层 gRPC 框架
会自动帮你处理数据序列化、网络通信、身份验证、访问控制、可观察性等远程通信相关的所有复杂的工作
server 端实现了一系列可供远程调用的函数，client 端可以
生成一个 stub，它提供一系列与 server 端远程函数一一对
应的函数，client 端可以直接调用 stub 提供的函数，就像
调用远程函数一样。
图为 client 端调用远程函数的示例。server 端提供
getProduct 函数来获取指定 productID 的产品详情，
client 端调用 stub 提供的对应的 getProduct 函数，来调
用远程函数。整个调用过程如下
1. client 端调用 stub 中的 getProduct 函数；
2.
client stub 对 message 进行编码，并创建一个 HTTP POST 请求。在
gRPC 中，所有请求都是 HTTP POST 请求，content-type 为
application/grpc。它调用的远程函数 (/ProductInfo/getProduct) 会作为
单独的 HTTP 标头（path）发送；
3. HTTP POST 请求通过网络发送到 server 端；
4. 当 server 端接收到 message 时，它会检查 HTTP 头，确认需要调用哪个
server 函数，并将 message 转交给 server stub；
5. server stub 将 message 解码为指定的数据结构；
6. 之后，server 端本地调用 getProduct，并将解码后的 message 作为参数传
递给该函数。
7.
server 端函数返回结果后，会对结果进行编码并发送回 client 端。响应结果
的处理过程和 client 端发送 message 的处理过程一致（响应结果→编码
→HTTP 响应）；
8. client 端收到 message 之后，将其解码，并传给等待的 client 进程。
这些步骤与大多数 RPC 系统（如 CORBA、Java RMI 等）非常相似。gRPC 与它们之间的主要区
别在于它对 message 进行编码的方式，它使用 protocol buffer 进行编码。
Grpc适用场景
微服务：gRPC 设计用于低延迟和高吞吐量通信。 gRPC 对于效率至关重要的轻量级微服务非常有用。
点对点实时通信：gRPC 对双向流式传输提供出色的支持。 gRPC 服务可以实时推送消息而无需轮询。
多语言环境：gRPC 工具支持所有常用的开发语言，因此，gRPC 是多语言环境的理想选择。
网络受限环境：gRPC 消息使用 Protobuf（一种轻量级消息格式）进行序列化。 gRPC 消息始终小于等效的 JSON 消息。
进程间通信 (IPC) ：IPC 传输（如 Unix 域套接字和命名管道）可与 gRPC 一起用于同一台计算机上的应用间通信。
QA
Protocol Buffers和gRPC什么关系
gRPC是RPC协议是一种实现，是一个框架；Protocol Buffers，是Google公司开发的一种数据描述语言
gRPC和Protocol Buffers的关系就好比浏览器和HTML的关系，不相互依赖，但是需要相互配合使用
Http2为什么不能代替 Grpc
rpc的元数据的传输不够高效尽管HPAC可以压缩HTTP Header，但是对于rpc来说，确定一个函数调用，可以简化为一个int，只要两端去协商过一次，后面直接查表就可以
了，不需要像HPAC那样编码解码。可以考虑专门对gRPC做一个优化过的HTTP/2解析器，减少一些通用的处理，感觉可以提升性能。
TODO
同步与异步? gprc支持同步和异步调用
Http2和rpc关系
服务发现 HTTP 通过 DNS 服务去解析 IP 地址，默认 80 端口；RPC中间服务去保存服务名和 IP 信息，比如 consul 或者 etcd，
甚至是 redis.dns 也是服务发现的一种，所以也有基于 dns 去做服务发现的组件，比如 CoreDNS。
底层连接形式 RPC 协议一般还会再建个连接池，在请求量大的时候，建立多条连接放在池内，要发数据的时
候就从池里取一条连接出来，用完放回去，下次再复用
传输的内容
http/1.1之前，消息头 header 和消息体 body是基于文本的，不高效，所以才有了rpc；同
时，RPC的概念80 年代提出，比HTTP出现得要早。Http2是2015年才出的
一个数据帧可能有很多 gRPC 消息，或者如果一个 gRPC 消息非常大它可能跨越多个数据帧。
RPC 本质上不算是协议，而是一种调用方式，而像 gRPC 和 thrift 这样的具体实现，才是协议，它们是实现了
RPC 调用的协议。同时 RPC 有很多种实现方式，不一定非得基于 TCP 协议
Dubbo
什么是Dubbo
Apache Dubbo 是一款 RPC 服务开发框架，用于解决微服务架构下的服务治理与通信问题，官方提供了 Java、Golang 等多语言 SDK
实现。使用 Dubbo 开发的微服务原生具备相互之间的远程地址发现与通信能力，利用 Dubbo 提供的丰富服务治理特性，可以实现诸如
服务发现、负载均衡、流量调度等服务治理诉求。Dubbo 被设计为高度可扩展，用户可以方便的实现流量拦截、选址的各种定制逻辑
在云原生时代，Dubbo 相继衍生出了 Dubbo3、Proxyless Mesh 等架构与解决方案，在易用性、超大规模微
服务实践、云原生基础设施适配、安全性等几大方向上进行了全面升级。
开源故事
2008 年，阿里巴巴就将 Dubbo 捐献到开源社区
2017 年，Dubbo 被正式捐献 Apache 软件基金会
2022 年双十一结束，Dubbo3 已经在阿里巴巴内部广泛落地，实现了老版本 HSF2 框
架升级，包括电商核心、阿里云等核心系统已经全面运行在 Dubbo3 之上
三大核心能力
面向接口的远程方法调用
智能容错和负载均衡
服务自动注册和发现
Dubbo架构
调用流程
服务容器负责启动，加载，运行服务提供者。
服务提供者在启动时，向注册中心注册自己提供的服务。
服务消费者在启动时，向注册中心订阅自己所需的服务。
注册中心返回服务提供者地址列表给消费者，如果有变更，注册中心将基于长连接推送变更数据给消费者。
服务消费者，从提供者地址列表中，基于软负载均衡算法，选一台提供者进行调用，如果调用失败，再选另一台调用。
服务消费者和提供者，在内存中累计调用次数和调用时间，定时每分钟发送一次统计数据到监控中心。
模块
dubbo-common:公共逻辑模块: 包括Util类和通用模型
dubbo-remoting 远程通信模块: 相当于dubbo协议的实现，如果RPC使用RMI协议则不需要使用此包
dubbo-rpc 远程调用模块: 抽象各种协议，以及动态代理，包含一对一的调用，不关心集群的原理。
dubbo-cluster 集群模块: 将多个服务提供方伪装成一个提供方,包括负载均衡,容错,路由等,集群的地
址列表可以是静态配置的,也可以是注册中心下发的.
dubbo-registry 注册中心模块: 基于注册中心下发的集群方式,以及对各种注册中心的抽象
dubbo-monitor 监控模块: 统计服务调用次数,调用时间,调用链跟踪的服务.
dubbo-config 配置模块: 是dubbo对外的api,用户通过config使用dubbo,隐藏dubbo所有细节
dubbo-container 容器模块: 是一个standlone的容器,以简单的main加载spring启动,因为服务通常不
需要Tomcat/Jboss等web容器的特性,没必要用web容器去加载服务.
各层说明
Dubbo的SPI
JDK中的SPI “基于接口的编程＋策略模式＋配置文件”组合实
现的动态加载机制。
Dubbo的SPI如何实现IOC和AOP的
Dubbo中的动态编译
QA
有了Dubbo协议，为什么还要引入Triple协议？ Dubbo不够通用，dubbo协议一旦涉及到跨RPC框架，比如一个Dubbo服务要调用gPRC服务，就比较麻烦了，因为发一个
dubbo协议的请求给一个gPRC服务，gPRC服务只会按照gRPC的格式来解析字节流，最终肯定会解析不成功的。
对比一下triple、dubbo、rest这三个协议
triple协议基于的是HTTP2，rest协议目前基于的是HTTP1，都可以做到跨语言。
triple协议兼容了gPRC（Triple服务可以直接调用gRPC服务，反过来也可以），rest协议不行
triple协议支持流式调用，rest协议不行
rest协议更方便浏览器、客户端直接调用，triple协议不行（原理上支持，当得对triple协议的底层实现
比较熟悉才行，得知道具体的请求头、请求体是怎么生成的）
dubbo协议是Dubbo3.0之前的默认协议，triple协议是Dubbo3.0之后的默认协议，优先用Triple协议
dubbo协议不是基于的HTTP，不够通用，triple协议底层基于HTTP所以更通用（比如跨语言、跨异构系
统实现起来比较方便）
dubbo协议不支持流式调用
其它协议
triple协议
Thrift
GraphQL
是什么
是 API 的一种查询语言，允许客户端来决定他们
想要什么数据、他们想要怎么获取数据以及他们
想要什么格式的数据
GraphQL 更适合直接面向外部的服务或 API，其
中客户端需要对从服务器获取的数据进行更多控
制。
RPC间的比较
gRPC和Thrift
传输
gRPC 并未给网络 I/O 提供抽象，而是强依赖于
HTTP/2 。它基于 HTTP/2 实现了传输的高效，
并支持流式传输等消息传递模式
流式传输 gRPC 原生支持双向流（客户端和服务器）
性能
子主题 2
gRPC和Dubbo
如何设计一个简单的RPC框架
扩展
SOA
SOA和**区别
1. 康奈尔笔记法
日期
基本问题
主题
笔记
问题
概要
QA